

PetPal

My Python notebook · 4 sessions · one electronic pet

This belongs to:

My pet's name will be:

My pet is a **cat / dog** (circle one)



draw your pet's colour here

PetPal your code, alive

Your code

```
31     pet.say("I am starvin
32     pet.eat()
33
34     # A loop + an if statemen
35     for year in range(4):
36         pet.birthday()
37         if pet.age <= 3:
38             pet.grow(0.25)
39         else:
40             pet.say("All grow
41
42     # A list of tricks
43     tricks = ["spin", "jump",
44     for trick in tricks:
45         brag(pet, trick)
46         if trick == "spin":
47             pet.spin()
48         elif trick == "jump":
49             pet.jump()
50         else:
51             pet.dance()
52
53     pet.turn()
54     pet.walk(2)
55     pet.cheer()
56     pet.say("Your turn. Let's
57
58
59 run(main)
```

Biscuit says something
Variables

name	str	'Biscuit'
color	str	'golden'
age	int	5
size	float	1.3
is_hungry	bool	False

Console - print() lands here

Replay **Slower** **Normal** **Faster** speed 4x

Your code

The yellow line is the line Python is running *right now*. Watch it move.

Your pet

Everything it does, you told it to do. The stones are numbered from 0.

Variables & Console

Every variable, with its **type**. `print()` lands underneath.

MY PROGRESS

- ☐ **Session 1** — I made a pet, gave it a name, an age and a size
- ☐ **Session 2** — I taught it to make decisions
- ☐ **Session 3** — I made it repeat things with loops
- ☐ **Session 4** — I wrote my own functions and built my own program

Two rules for the whole course

- Before you press Run, say out loud what you think will happen.
- When something breaks — and it will — **read the message**. That's not failure, that's the job.

Cheat sheet

Keep this one on the table. All four sessions.

THE FOUR TYPES

str	"Mochi"	text — always in quotes
int	2	whole number
float	1.4	number with a dot
bool	True False	yes / no
list	["a", "b"]	many things in one box

SAYING THINGS

```
print("goes to the Console")
pet.say("goes in a speech bubble")
pet.say("Hi " + pet.name)
pet.say(f"I am {pet.age} years old")
```

MAKING DECISIONS

```
if pet.age < 3:
    pet.grow(0.3)
elif pet.age < 10:
    pet.say("grown up")
else:
    pet.sleep(2)
```

== same? != different? < > <= >= · join with and or not

REPEATING THINGS

```
for i in range(4):      # 0 1 2 3
    pet.step()

while energy > 0:
    pet.jump()
    energy = energy - 1
```

YOUR PET CAN BE

```
pet.name = "Mochi"
pet.age = 2
pet.size = 1.4
pet.color = "lavender"
pet.is_hungry = True
pet.mood = "sleepy"
```

YOUR PET CAN DO

```
pet.say("hi")      pet.think("hmm")
pet.step()         pet.walk(3)
pet.back(2)        pet.turn()
pet.jump()         pet.spin()
pet.dance()        pet.wag()
pet.cheer()        pet.sit()
pet.eat()          pet.sleep(2)
pet.wake()         pet.wait(1)
pet.birthday()     pet.grow(0.2)
pet.shrink(0.1)
```

HELPERS

```
show("treats", 5)
name = ask("What's my name?")
age = int(ask("How old?"))
help_me()
```

FUNCTIONS AND LISTS

```
def celebrate(pet):
    pet.jump()
    pet.cheer()

tricks = ["jump", "spin"]
for t in tricks:
    print(t)
tricks.append("sit")
```

When it breaks, check these three first

1. Quotes — text needs "quotes", numbers don't **2. Colon + indent** — after if for while def **3. Spelling** —
pet.Say is not pet.say

Session 1 — Meet Your Pet

variables · str · int · float · bool

A variable is a labelled box

You put something in the box, and the label is how you get it back later. `pet.name = "Mochi"` means *put the text Mochi in the box called name*.

1. FILL IN THE TYPE OF EACH ONE

CODE	TYPE	HOW YOU KNEW
<code>pet.name = "Luna"</code>		
<code>pet.age = 7</code>		
<code>pet.size = 0.5</code>		
<code>pet.is_hungry = False</code>		
<code>pet.color = "mint"</code>		
<code>pet.age = "7"</code>		

The last one is a trap. Why is it different from the second one?

2. PREDICT, THEN RUN

Write your prediction *before* you press Run. Being wrong is useful.

I WILL CHANGE	I PREDICT	WHAT REALLY HAPPENED
<code>pet.size = 2.0</code>		
<code>Pet("cat")</code>		
<code>pet.color = "purple"</code>		

3. THE FAMOUS ERROR

Type this in, and run it:

```
pet.say("I am " + pet.age + " years old")
```

Copy what the Console says, in your own words:

Now fix it two different ways:

4. BOSS CHALLENGE

Make your pet walk to stone 4, jump, turn around, walk back to stone 1, and say goodbye using its own name. Write your code here first:

Session 2 — Pet Decisions

comparisons · if / elif / else · and / or / not · ask()

The most confusing pair in programming

= means **put this in the box** (pet.age = 3)

== means **are these the same?** (if pet.age == 3:)

1. TRUE OR FALSE?

Pretend age = 4 and is_hungry = True.

age > 2

age == 4

age != 4

age <= 3

is_hungry

not is_hungry

age > 2 and is_hungry

age > 9 or is_hungry

2. WHICH LINE RUNS?

```
if pet.mood == "excited":  
    pet.dance()           # A  
elif pet.mood == "sleepy":  
    pet.sleep(2)          # B  
else:  
    pet.wag()             # C
```

IF PET.MOOD IS...

WHICH ONE RUNS? (A / B / C)

"sleepy"

"excited"

"grumpy"

"Sleepy" (capital S!)

The rule: Python reads top to bottom, takes the **first** one that is True, and then **stops reading the rest**.

3. THE FEEDING MACHINE

Ask how many hours since the last meal. Then: more than 6 → very hungry (say it, eat, cheer); 3 to 6 → eat; under 3 → not hungry, dance instead. Plan it here:

4. WHY DOES THIS NEED INT()?

```
answer = ask("How many treats?")  
treats = int(answer)
```

Session 3 — Loops

for · range() · counters · while · loops inside loops

Why loops exist

Computers are fast but not clever. A loop is how you say "do that again" instead of copying and pasting the same line ten times.

1. WHAT NUMBERS COME OUT?

range(4)

range(1, 5)

range(0, 10, 2)

range(5, 0, -1)

range(3)

range(2, 3)

2. TRACE IT — WHERE DOES THE PET END UP?

It starts on stone 0 facing right.

```
for i in range(3):  
    pet.step()  
pet.turn()  
for i in range(2):  
    pet.step()
```

```
for a in range(2):  
    for b in range(3):  
        pet.jump()  
        pet.spin()
```

Final stone: _____

Jumps: _____ Spins: _____

3. THE GROWTH SIMULATOR

Fill in the missing pieces so the pet has 5 birthdays but only grows while it is 3 or younger.

```
for year in range(1, _____):  
    pet._____( )  
    if pet.age ____ 3:  
        pet._____(0.25)  
    else:  
        pet.say("I stopped growing!")
```

How many birthdays happen? _____ How many times does it grow? _____ Why aren't those the same number?

4. THE INFINITE LOOP

```
energy = 3  
while energy > 0:  
    pet.jump()
```

This one never stops. Why — and what one line fixes it?

5. BOSS CHALLENGE — THE PET OLYMPICS

At least 3 loops and 1 if. Must include a countdown, a walk, a dance and a victory cheer. Sketch it:

Session 4 — Functions, Lists & My Own Program

def · parameters · return · lists · random

A function is a named box of steps

Write the steps once, use them as many times as you like. **Defining a function does not run it** — nothing happens until you *call* it.

1. LABEL THE PARTS

```
def do_trick(pet, trick_name):
    pet.say("Watch this: " + trick_name)
    pet.jump()

do_trick(my_dog, "backflip")
```

The name of the function is _____

Its parameters are _____

The line that *calls* it is _____

Lines that are *inside* it are _____

2. DOING VS ANSWERING

	DOES SOMETHING	GIVES AN ANSWER BACK
pet.jump()		
len(tricks)		
int("7")		
print("hi")		

The keyword that hands an answer back is: _____

3. LISTS

tricks = ["jump", "spin", "dance"]	
tricks[0] is _____	len(tricks) is _____
tricks[2] is _____	tricks[3] gives _____

4. MY FINAL PROJECT PLAN

My program is called: _____

In one sentence, it: _____

It must contain all of these:

- ☐ 3+ variables, 2+ types
- ☐ an if / elif / else
- ☐ a loop
- ☐ a function I wrote myself
- ☐ a list
- ☐ an ask() so a human can play

The smallest version that would still work: (build this first, then add)

Certificate of Programming

awarded to

who wrote real Python code, made a computer do exactly what she told it to (and sometimes what she didn't), read her own error messages, and built a program that nobody had ever written before.

She can now use:

variables · str, int, float, bool
if / elif / else · and, or, not
for loops · while loops · range()
functions · parameters · return
lists · random · input

Her final project was:

.....
.....
.....

teacher

date

Where to go next

- **Keep your pet.** `my_pet.py` is yours. Nothing in it is homework.
- **Python Crash Course** by Eric Matthes — the classic first book.
- **Turtle graphics** — already inside Python. Then **pygame** for real games.
- **Girls Who Code** clubs · **Technovation** · local hackathons.
- **Automate the Boring Stuff with Python** — free to read online.